

Soraya Unified Architecture v0.4

Canonical specification. Working document. Prototype stage. Human-reviewed validation required before operational deployment.

1. Thesis

External: Soraya is an agency-preserving AI governance system that controls what context may enter, what judgment may be offered, what action may execute, and whether the human becomes more capable over time.

Internal: Soraya is an authorization-aware, judgment-routed, least-resonance human-AI governance system that binds context access, epistemic legitimacy, policy enforcement, trajectory stability, and agency growth into a measurable control architecture.

Both are true. The external version sells. The internal version specifies.

2. Design Invariants

The four-question gate. No layer is added unless it answers one of:

1. What context may enter?
2. What judgment may be offered?
3. What action may execute?
4. Did the human become more capable?

Anything else is probably whitepaper smoothie.

The Route/Disposition invariant. This is the most important safety rule in the architecture:

Reviewer saturation may change execution disposition. Reviewer saturation may not downgrade epistemic or safety classification.

Formally:

Route(x) – epistemic/safety classification
Stable. Not capacity-aware. Never lowered by queue pressure

Disposition(x, H_t) – execution decision
Capacity-aware. May be deferred, blocked, or escalated based on H_t, but never reclassified to a lower route.

The failure mode this prevents: queue pressure becoming implicit permission. If H_t is saturated, Route(x) = DEEP must remain DEEP. Disposition changes; classification does not.

The non-erasure rule. Risk-mitigating features (evidence_available) may not be raised by semantic inference alone. Actual source verification required.

The memory-authority rule. Memory never grants authority. Retrieved relevance is subordinate to authorization.

The enforcement rule. Ledger flags are not enforcement. A governance router that merely says “bad idea” is a hallway monitor. Enforcement requires a Policy Enforcement Point that physically blocks action at the tool/API/retrieval boundary.

The S(x) constraint rule. The executive-function support layer may shape support style. It may not lower a safety route. It may not grant authority.

3. Authorized Input Object: x_t

Before x is constructed, the Context Authorization Layer determines what may enter it.

x_t = (u, c_authorized, h_consented, s_permitted, τ)

Symbol	Component	Authorization requirement
u	User message	Identity verified
c_authorized	Conversation context	Role-authorized documents only
h_consented	Session history	User-consented memory only
s_permitted	System state	Tool/API permissions verified
τ	Trust context	Source trust zone, role, policy version, action scope

Where:

```
 $\tau = (\text{role\_id}, \text{source\_trust\_zone}, \text{policy\_version}, \text{action\_id},$   
       $\text{permission\_level}, \text{tool\_allowed}, \text{approval\_required}, \text{audit\_required})$ 
```

Authorization-first retrieval. Retrieve-then-filter is the wrong order. Once restricted data enters model context, the boundary has already been crossed. No document, memory, HR record, ticket, or tool output enters context unless the user/session is authorized to see it.

Authorization-first RAG rule:

```
authorized_context = {d ∈ retrieved_documents : can_access( $\tau$ .role_id, d)}  
c_authorized = authorized_context      // not: retrieve_all → filter later
```

4. Judgment Router: $J(x)$

What $J(x)$ answers: What kind of judgment does this situation deserve?

Feature Extraction Layer

```
 $\phi(x) = \text{keyword\_features}(x)$                 [always]  
      ⊕  $\text{semantic\_features}(x)$                 [DEEP path only, when available]
```

Merged via confidence-gated conservative max:

```
 $f_i(x) = \max(\phi_{\text{keyword}_i}, \phi_{\text{semantic}_i})$    if confidence( $\phi_{\text{semantic}_i}$ ) ≥  $\tau_{\text{confide}}$   
      =  $\phi_{\text{keyword}_i}$                            otherwise
```

Components:

- Deterministic keyword detector (v0.8 frozen baseline)
- Semantic gap-case similarity detector (specification phase)
- Confidence-gated merge
- Non-erasure rule

Epistemic Activation and Profile

$$a_i(x) = \min(\sum_k \delta_{ik}(\phi(x)), CAP_i) \quad [\text{evidence accumulation, may exceed 1.0}]$$

$$I(x) = \sum_i a_i(x) \quad [\text{total intensity - L1 norm}]$$

$$p_i(x) = a_i(x) / I(x) \quad \text{when } I > 0 \quad [\text{normalized profile - shape}]$$

$p(x)$ = what kind of situation. $I(x)$ = how much is happening simultaneously. Never normalize this away.

Risk Scores

$$\begin{aligned} \rho(x) = & w_1 \cdot \text{uncertainty} + w_2 \cdot \text{high_stakes} + w_3 \cdot \text{authority} \\ & + w_4 \cdot \text{contradiction} + w_5 \cdot \text{dependency} + w_6 \cdot \text{irreversibility} \\ & - w_7 \cdot \text{evidence_available} \quad [\rho \in [0,1]] \end{aligned}$$

$$\lambda(x) = P(\text{fluent answer} \equiv \text{adequate grounds for action}) \quad [\lambda \in [0,1]]$$

Forbidden Modes and Lead Mode

$$F(x) = \{r \in R : (C \cdot p(x))_r \geq \tau_r\} \quad [\text{structural constraint}]$$

$$r^*(x) = \operatorname{argmax}_{\{r \notin F(x)\}} U(r, x)$$

$$U = \alpha \cdot \text{epistemic_fit} + \beta \cdot \text{helpfulness} - \gamma \cdot \text{false_certainty_risk} - \delta \cdot \text{agency_displacem}$$

Priority: Safety > Authority > Agency > Task usefulness > Fluency

Constraints

$A_{\text{agency}}(x)$: preserve_user_ownership | avoid_dependency_reinforcement |
return_next_action_to_user | offer_options_not_decision

$A_{\text{process}}(x)$: verify_authority_before_action | require_human_review |
escalate_to_professional | cite_evidence | verify_sources

A_{agency} protects the human. A_{process} protects the workflow. Orthogonal. Both required.

J(x) Output Tuple

$$J(x) = (a, p, I, \rho, \lambda, r^*, F, A_{\text{agency}}, A_{\text{process}}, d, G)$$

5. Governance State: G_t with H_t

The governance state is maintained by analysts, not inferred by the model alone:

$G_t = \{A_t, S_t, P_t, Q_t, E_t, H_t, W_t, \Theta_t\}$

Component	Content
A_t	Role and authority map
S_t	Source trust zones
P_t	Action sensitivity classification
Q_t	Phrase/pathway trigger map
E_t	Escalation policy
H_t	Human oversight capacity (see below)
W_t	Scoring weights
Θ_t	Routing thresholds

H_t — Human Review Capacity

```
H_t = {
  reviewer_capacity:  active reviewers available,
  queue_depth:       pending items in review queue,
  queue_age:         median age of queue items (hours),
  fatigue_factor:    reviewer workload signal [0,1],
  severity_mix:      proportion of DEEP vs MEDIUM in queue,
  turnaround_sla:    target review window
}
```

Critical rule: H_t affects Disposition, not Route. When H_t is saturated:

```
Route(x)    = DEEP          // unchanged
Disposition = block | defer | emergency_escalate // changed

NOT:
Route(x) = MEDIUM          // WRONG — safety theater with an excuse
```

Saturation states and their dispositions:

H _t state	Allowed dispositions
Healthy	allow, clarify, verify, require_review
Elevated	require_review, defer, verify
Saturated	defer, block, emergency_escalate
Critical	block, emergency_escalate only

6. Enforcement Output: $\Gamma(x, H_t)$

What Γ answers: Is this role permitted to perform this action, and given current capacity, what disposition is executable?

Authority Resolution

```
 $\Gamma_{auth}(x) = \text{lookup}(\tau.\text{role\_id}, \tau.\text{action\_id}) \rightarrow \text{permission\_level}$   
  
 $\text{authorized}(x) = \text{permission\_level} \geq \text{required\_level}(\tau.\text{action\_id})$   
                  AND (NOT approval_required OR human_review_obtained)
```

Capacity-Aware Disposition

```
 $\Gamma(x, H_t) \in \{$   
    allow,  
    clarify,  
    verify,  
    require_review,  
    defer,  
    block,  
    emergency_escalate  
 $\}$ 
```

Selection logic:

```
if NOT authorized(x):  
     $\Gamma = \text{block} \mid \text{emergency\_escalate}$ 
```

```

elif Route(x) == DEEP and H_t == saturated:
     $\Gamma$  = defer | block    // never: reclassify to MEDIUM

elif Route(x) == DEEP:
     $\Gamma$  = require_review

elif Route(x) == MEDIUM:
     $\Gamma$  = verify | require_review

else: # FAST
     $\Gamma$  = allow | clarify

```

Policy Enforcement Point

The PEP enforces $\Gamma(x, H_t)$ physically at the tool/API/retrieval boundary. $\Gamma = \text{block}$ must prevent execution. Not flag it. Not log it. Prevent it.

Ledger flags are not enforcement. A PEP is infrastructure.

7. Soraya Executive-Function Support: $S(x)$

Selected only after authorization and governance routing have completed.

```

 $S(x) \in \{$ 
    clarify,
    triage,
    tiny_step,
    reset,
    reflection,
    momentum,
    verify,
    escalate,
    refuse
 $\}$ 

```

Hard constraints on $S(x)$:

- May shape support style and framing
- May not lower a safety route
- May not grant authority
- May not override $F(x)$ forbidden modes

- May not reduce `A_process` verification requirements

`S(x)` is downstream of governance. It is not a bypass of governance.

8. Least-Resonance Trajectory Monitor: LR(H)

Monitors conversation trajectory across turns, not just single-turn safety.

```
LR(H) = trajectory_analysis(conversation_history H)
```

```
Output: {  
  health_score:          [0,1],  
  detected_patterns:     list,  
  intervention_trigger:  bool,  
  recommended_mode:      r* override if critical  
}
```

Detected patterns:

Pattern	Signal
Dependency spiral	Increasing <code>dependency_risk</code> across turns; user agency score declining
Reassurance loop	Repeated requests for validation on same decision
Authority collapse	User delegating progressively larger decisions
Over-explanation loop	System generating longer responses with no user action
Avoidance loop	Recurring topic with no resolution or user-initiated closure
Repetitive attractor	Prompt-response pattern repeating without progress

Intervention rule: When `LR(H).intervention_trigger = True`, the Judgment Router receives a trajectory override signal that may elevate `dependency_risk` and force `avoid_dependency_reinforcement` as the agency constraint, regardless of single-turn classification.

What LR monitors that J(x) cannot: `J(x)` classifies individual turns. `LR(H)` detects whether the interaction as a whole is producing growth or reinforcing stasis. These are different measurements.

9. Agency and Process Ledger: L(x)

Operationalizes the recursive ethical loop as measurable outputs.

```
L(x) = {
  entry_id, timestamp, prompt_hash,

  context_authorization: {
    role_verified, documents_authorized, memory_consented,
    tool_permissions_verified, policy_version
  },

  triage: {
    route:          Route(x),          // stable, not capacity-adjusted
    bottleneck_score: B(x),
    components:      {A, Rs, K, T, U},
    weights_hash:    "...",
  },

  judgment: {
    epistemic_profile:  p(x),
    raw_activation:     a(x),
    total_intensity:    I(x),
    epistemic_risk:     ρ(x),
    unsafe_reliance_risk: λ(x),
    lead_mode:          r*(x),
    forbidden_modes:    F(x),
    agency_constraint:  A_agency(x),
    process_constraint: A_process(x),
    known_limitations:  [...]
  },

  enforcement: {
    disposition:        Γ(x, Ht),
    authorized:          bool,
    h_t_state:           healthy | elevated | saturated | critical,
    prompt_constraints_injected: bool,
    compliance_status:   passed_gross | failed | not_checked,
    compliance_limitations: [...]
  },

  trajectory: {
    health_score:        LR(H).health_score,
    detected_patterns:   [...],
    intervention_fired:  bool
  }
}
```

```

    },

    agency: {
        final_decision_owner:    system | user | reviewer,
        user_action_required:    bool,
        system_action_taken:     str,
        dependency_risk_score:    float,
        user_agency_score:       float,
        agency_delta_estimate:    float
    },

    final_status:
        passed_no_review_required |
        passed_review_required |
        blocked_or_deferred,

    provenance: {
        versions_and_hashes: {...},
        entry_hash:          "SHA-256(canonical_json(L without entry_hash))",
        previous_entry_hash: "...",      // tamper-evident chain
    }
}

```

The four ethical loop questions, operationalized:

Question	Ledger field
Did this preserve agency?	agency.final_decision_owner , agency.user_agency_score
Did this reduce dependency?	agency.dependency_risk_score , agency.agency_delta_estimate
Did this obey judgment boundaries?	enforcement.compliance_status , judgment.forbidden_modes
Did this follow the right process?	enforcement.disposition , enforcement.authorized

10. HITL Console and Known-Limitations Register

Limitations are promoted from test artifacts to operational dashboard features.

Visible to analysts:

Panel	Content
Detector gaps	Unresolved known-limitation cases (JR-013, JR-029, ...)
Evidence tiers	Which labels are strong / weak heuristic / external-signal-only
Threshold status	Current θ values, calibration date, holdout lock status
Queue burden	H_t : queue depth, age, severity mix, SLA status
Reviewer saturation	Current H_t state and disposition constraints in effect
Failure examples	Representative cases from each known gap category

Operating rule: Weak evidence is never hidden. It is displayed as a warning tile. A known limitation that is invisible is not a known limitation — it is a silent failure waiting for production.

11. Scenario Pack Evaluation Harness

Formal evaluation layer. Not a demo surface. A repeatable proof system.

Packs:

learn.json	– tutoring / knowledge support
work.json	– executive function, task management
onboard.json	– HR process, role clarity
govern.json	– enterprise governance, policy routing
dependency_cases.json	– agency preservation under dependency pressure
authority_confusion.json	– routing under authority confusion signals
safety_gap_cases.json	– known-limitation coverage

Success criteria per pack:

Pack	Primary success criterion
learn	Supports understanding without outsourcing the answer
work	Reduces friction without absorbing decision ownership
onboard	Clarifies role without making employment decisions
govern	Escalates correctly; does not auto-approve

dependency	Agency score non-declining across session
authority	Routes correctly without over-escalating on benign queries
safety_gap	Known limitations visible; desired behavior documented

The agency growth measurement: `agency_delta_estimate` tracked across scenario pack sessions. The architecture succeeds if users become demonstrably more capable over time — not more reliant.

12. Future Work: Memory Governance Specification

Memory governance is not included in v0.4. It is flagged as a mandatory companion specification.

The problem: Soraya will eventually require persistent memory for executive-function support — goals, routines, recurring blockers, workplace context. Memory is also where helpfulness quietly becomes surveillance.

Constraints for the memory governance spec:

- User-consented memory only
- Explicit forget/revise mechanism with TTL
- Separate personal memory from organizational policy
- No inferred sensitive attributes stored as operating facts
- Memory never grants authority (memory-authority rule)
- Memory subject to the same Context Authorization Layer as retrieved documents
- Cross-session state management without cross-session dependency amplification

This is where the dragons are wearing productivity-app hoodies. Memory governance is a first-class safety concern, not a product polish task.

13. Kaleidoworks Governance Family

Soraya is one member of a family of governance architectures that share the core principle:

Semantic relevance is not operational authority.

System	Domain	Core question
Soraya	Human agency / executive-function	What judgment may be offered? Does the human grow?
C3-ICM / LRAI	Connotative governance spine	Is this input authorized to activate this pathway?
Agentic RAG	Enterprise retrieval / access control	What context may enter?
Genesis	Cyber-physical / energy authority	Does this computation have authority to draw this resource?

These are not one product. They are one principle, instantiated across domains.

For external communication: Soraya is the human-facing layer. The others are infrastructure. For internal architecture: they share the ledger model, the Route/Disposition split, and the non-erasure rule.

Summary: The Unified Pipeline

```
 $\Pi(x\_t)$ :
# Stage 0a – Context Authorization
x_t ← authorize_context(u, role, memory_consent, tool_permissions)

# Hard-stop preflight
if preflight(x_t) == BLOCK:
    ledger.record_block(x_t)
    return boundary_response

# Stage 1 – Feature Extraction
 $\phi$  ← extract_features(x_t)      # keyword + semantic (DEEP only)

# Stage 2 – Judgment Routing
J ← judgment_router(x_t,  $\phi$ )
Route ← J.route                  # FAST | MEDIUM | DEEP – stable

if Route == FAST_PATH:
    Disposition ← allow
else:
```

```

# Stage 3 – Governance + Capacity-Aware Disposition
B ← bottleneck_score(x_t)
H_t ← get_reviewer_capacity()
Γ ← authority_resolution(x_t, J, H_t)
Disposition ← Γ.disposition # never reclassifies Route

if Disposition in {block, emergency_escalate}:
    ledger.record_full(x_t, J, Γ, Route, Disposition)
    return boundary_response

# Stage 4 – Executive Function Support
S ← support_mode(x_t, J) # constrained: may not lower Route

# Stage 5 – Trajectory Monitor
LR ← trajectory_monitor(conversation_history)
if LR.intervention_trigger:
    J ← apply_trajectory_override(J, LR)

# Stage 6 – Response Generation
prompt ← build_prompt_constraints(J, S)
output ← generate(prompt, x_t)

# Stage 7 – Compliance Check
compliance ← check_forbidden_modes(output, J)
if compliance.failed:
    output ← regenerate_or_escalate(prompt, x_t, J)

# Stage 8 – Ledger
ledger.record_full(x_t, φ, J, Route, Disposition, Γ, LR, S, output, complianc
return output

```

Governing Principle

Do not optimize for the most fluent answer.

Do not optimize for the fastest authorized response.

Optimize for the most legitimate reasoning mode for the situation,
under verified authority, with human judgment preserved
at every threshold where stakes exceed the system's epistemic warrant,
and with measurable evidence that the human became more capable.

v0.4 canonical specification. Requires human-reviewed validation before operational deployment. Memory governance specification pending as companion document.